

Planning pour le module R1.03 (Architecture)

D. Roegel

20 août 2026

Préambule : vers le cours parfait à l'ère de l'IA

Avant de détailler le planning du cours d'architecture, j'aimerais vous transmettre quelques réflexions sur les objectifs du cours, sur l'articulation avec l'IA et sur la manière dont j'entrevois la résolution des problèmes qui se posent. Ces réflexions ne sont aucunement définitives et toute suggestion sera bienvenue pour les améliorer.

Mon objectif est de déterminer l'organisation du cours comme la solution d'un problème. Il faut donc d'abord poser précisément ce problème.

1. Je pars du principe que le cours d'architecture est donné. Il ne s'agit pas ici de décider s'il faut ou non enseigner l'architecture à nos étudiants. Un jour peut-être on n'enseignera plus l'architecture, ni le codage, ni l'algorithmique, ni Java, etc., et on apprendra uniquement à nos étudiants à créer des applications avec des *prompts*.
2. Avant de voir comment le cours d'architecture peut s'organiser, il faut, je crois, fixer les objectifs à la fois pour les étudiants et pour les enseignants (on oublie souvent ces derniers!).
3. En ce qui concerne les étudiants, mon objectif est de leur inculquer des notions sur l'architecture des ordinateurs, le codage des informations, la mémoire, les opérations logiques, la représentation des nombres, le fonctionnement d'un processeur, les processus, etc. Il s'agit aussi d'en profiter pour évoquer certains aspects historiques et faire une petite initiation au C.
4. Pour atteindre cet objectif, il apparaît nécessaire d'éviter le recours à l'IA (intelligence artificielle). L'objectif du cours d'architecture n'est pas d'apprendre à utiliser l'IA, de faire du copier-coller, etc., mais de donner aux étudiants des connaissances de base, utilisables hors-IA. Cela pose toute la question de la connaissance, mais si la connaissance n'est plus importante, alors je pense que les études ne le sont plus non plus. En évitant l'usage de l'IA, on peut espérer donner aux étudiants une certaine maîtrise des sujets et de leur faire faire des choix plus éclairés. Par ailleurs, si les étudiants n'ont pas un certain nombre de connaissances de base, ils ne seront pas utiles dans les entreprises. L'IA fera mieux.
5. En même temps, je pense qu'il y a un autre objectif à intégrer, à savoir que le cours ne doit pas rendre la vie des enseignants plus difficile, plus fastidieuse, mais plutôt plus intéressante. Nous ne devons pas devenir des policiers et nous ne devons pas nous épuiser face à des étudiants qui eux utilisent toutes les

armes modernes possibles contre nous, sans se poser de questions. Nous devons réfléchir à notre rôle d'enseignant à l'ère de l'IA. Notre but doit être de satisfaire le premier objectif d'enseignement de connaissances, celui des étudiants, sans nous épuiser.

6. En ce qui concerne la forme du cours, je pense qu'il y a un certain nombre de choses qui ne fonctionnent plus, en tous cas avec nos étudiants et avec cette matière. On peut faire les constatations suivantes :
 - (a) un cours au tableau comme autrefois ne fonctionne plus, les étudiants n'ont pas envie de recopier ce qu'écrit l'enseignant, l'enseignant va se fatiguer pour rien, les étudiants feront autre chose dans son dos, etc. ;
 - (b) lire un support en le projetant est tout aussi dépassé ;
 - (c) le support d'architecture est destiné à être lu du début à la fin, mais il n'est pas destiné à être projeté et parcouru dans l'ordre ; tous ceux qui ont essayé de le faire (moi y compris) ont vu que cela ne va pas ; il faut au minimum sauter des parties, mais en fait on peut même sauter des parties pour y revenir plus tard.
 - (d) on ne peut pas surveiller les étudiants en permanence ou espérer que l'on contrôlera tout ;
 - (e) pour l'évaluation, il est devenu très difficile de lire l'écriture des étudiants qui ne savent plus écrire, nous perdons donc énormément de temps à parcourir des copies, sans que cela soit vraiment utile ni pour les étudiants (qui ne voient presque jamais leurs copies), ni pour nous.
7. Il faut donc trouver un meilleur moyen pour atteindre les objectifs du cours, sans que ce soit une contrainte anormale ni pour les étudiants, ni pour les enseignants.
8. Il faut aussi se demander quel est le rôle de l'enseignant à l'ère de l'IA. Pour moi le rôle de l'enseignant est aujourd'hui
 - (a) de guider les étudiants, de les orienter ;
 - (b) de les inciter à travailler ;
 - (c) de leur donner des pistes ;
 - (d) et puis de les évaluer au travers d'exercices.

Le rôle de l'enseignant n'est plus de faire un cours, surtout sur un sujet classique.
9. Toutes ces considérations déterminent la forme du cours que je propose, mais je suis preneur d'améliorations qui faciliteraient 1) la transmission des connaissances, 2) l'évaluation des étudiants, 3) la vie des enseignants.

1 Organisation du cours

1.1 Volume horaire

Le volume total par étudiant est de 14h TD + 8h TP + 2h DS, réparties sur quatre semaines.

Chaque semaine, en principe : 4h de TD + 4h de TP en 1/2 groupes, donc 2h par 1/2 groupe. On fait à-peu-près deux fois la même chose en TP et nous avons chacun 8h par semaine.

1.2 Parties théorique (TD) et pratique (TP)

Les séances se partagent entre une partie théorique (sans ordinateur) et une partie pratique. En principe, les salles de TD n'auront pas de machines et les étudiants n'auront droit ni à leur ordinateur, ni à leur téléphone.

Par contre, l'enseignant a le droit de projeter le support de cours, ou d'éventuels autres documents.

En TP, les étudiants ont droit à tout. Les TP sont en principe en demi-groupes (à vérifier), on fait donc à-peu-près deux fois la même chose.

Cette année, j'ai choisi de faire imprimer le support, pour que les étudiants puissent le feuilleter, l'annoter, etc., et surtout le parcourir sans ordinateur en TD. L'objectif est qu'ils puissent ainsi mieux se l'approprier, même si le PDF reste en ligne.

Je donne ci-dessous des généralités sur les TDs et les TP. Chaque TD et TP est encore détaillé plus loin.

1.3 Forme de la partie théorique (TDs)

Pour moi, le cours doit être réduit au minimum, disons au maximum 30 mn sur 2h. Le reste doit être consacré au travail personnel des étudiants et à leur évaluation, par des passages au tableau ou des rendus de travaux.

Le premier TD sera différent des suivants et doit être traité avec attention. Il doit proposer un panorama de tout le module. Il s'agit de parcourir tout le support, mais pas dans les détails, pour montrer aux étudiants où l'on va ; le premier TD peut aussi être l'occasion de montrer un peu l'environnement Linux, de faire un peu de C, de parler de codage, etc.

Il faut évidemment bien dire aux étudiants qu'on fait un parcours rapide, et que ce n'est pas grave s'ils ne comprennent pas tout. Le premier TD doit aussi expliquer aux étudiants la structure générale d'un TD, plus donner du travail pour le prochain TD. En l'occurrence, il s'agira de lire les chapitres 1, 2 et 3 dans le détail.

Les autres TDs auront la forme suivante :

- on commence par demander aux étudiants s'ils ont des questions sur les séances précédentes ou aimeraient revenir sur tel ou tel point ;
- on fait 30 mn de cours maximum, éventuellement fractionné, sur une partie qui devait être lue depuis la séance précédente ;
- on indique aux étudiants ce qui est à lire pour la prochaine fois (quels chapitres ou pages) ;
- on passe aux exercices, en indiquant un ordre précis (au choix des intervenants) ;
- par intermittence on fait passer des étudiants au tableau et on les note ;
- on interroge notamment aléatoirement certains étudiants au tableau sur le cours passé, qu'ils sont supposés avoir étudié ;
- on ramasse certaines copies, au hasard ou pas (au choix des intervenants).

1.4 Forme de la partie pratique (TPs)

En TP, on travaille sur machine, en faisant du C, en travaillant sur les codages, sur des simulateurs, etc. ; il n'y a pas particulièrement d'évaluations.

Les étudiants ont seulement besoin de pouvoir faire un peu de C, on peut cependant les encourager à installer Linux.

En semaine 3, la séance de TP est utilisée pour les tests oraux de 5mn sur IEEE754 et sur les portes logiques. Les étudiants travaillent sur les exercices prévus et viennent à tour de rôle se faire interroger. La note compte pour 5 points et sous forme additive. Cette note peut être modulée par les devoirs ramassés en TDs.

1.5 Forme de l'examen

Le problème de beaucoup de sujets d'examens est que les questions sont trop ouvertes. On se retrouve avec des copies illisibles, qui n'apportent quelque chose ni aux étudiants, ni à nous.

En ce qui me concerne, je ne veux pas lire l'écriture des étudiants. Je privilégie donc les QCM au maximum.

Si l'un d'entre vous a envie de lire de la prose d'étudiants, libre à lui de proposer un exercice dans ce sens, mais c'est lui ou elle qui le corrigera.

Le DS comportera plusieurs exercices qui seront corrigés par les différents intervenants. Si quelqu'un veut un exercice particulier, qu'il le propose. Sinon, je ferai un sujet dans le genre de ceux des années précédentes, avec des QCM partout.

2 Documents fournis

À part le présent document, il n'y a que le fichier du cours qui est en

`https://tmpdocs.github.io/r103/r103-cours.pdf`

Ce document est encore susceptible d'évoluer un petit peu, mais il est en principe presque finalisé.

Le lien est à donner à vos étudiants. Je ne mets rien sur ARCHE. Les supports sont déjà imprimés et seront disponibles fin août pour vous.

Il n'y a pas de fichiers séparés pour les TDs, TP, ni même de transparents de cours. Tout est en un. Projetez le support et agrandissez si nécessaire. Mais ne faites pas une présentation linéaire du support! Le support est destiné à être lu intégralement, pas à être projeté intégralement.

3 Évaluation

Il y a un DS à la fin, plus une note de participation (0 à 5), plus une évaluation orale au milieu du module (5mn sur un exercice).

Le DS pourra être composé d'exercices, de questions de cours et de QCM ; il faudra donc avoir lu tout le cours. Aucun document ne sera autorisé au DS, sauf éventuellement deux pages manuscrites, et les sacs devront être rassemblés à l'avant des salles d'examen (avec les téléphones, lunettes IA, montres connectées, oreillettes, implants, etc.).

La note de participation est basée sur le rendu d'exercices et l'assiduité. Chaque semaine, tous les étudiants devront rendre plusieurs exercices en TD sous forme *manuscrite* (voir liste plus loin). Les feuilles pourront leur être rendues en fin de module,

mais devront être conservées pour l'évaluation. Chaque enseignant juge les rendus à sa discrétion, mais au final il faut une note de 0 à 5. Les étudiants devront au moins faire l'effort d'écrire leur solution avec un stylo. Ces exercices doivent être individuels. N'hésitez pas à mettre 0 si des copies sont identiques.

Pour l'examen oral, le plus simple sera de le faire pendant le TP de la semaine 3. Vous serez dans votre bureau, vous ferez venir les étudiants toutes les 5mn, vous leur donnerez un sujet (que je vous donnerai, genre IEEE754) et ils vous l'exposeront en 5mn, sans préparation. Vous donnerez une note entre 0 et 5. Cette note sera un élément de la note finale (on verra comment on combine les notes).

4 Planning général

Je souhaite que l'on fasse deux ou trois chapitres par semaine :

- Semaine 1 : chapitres 1, 2, 3 et 10
- Semaine 2 : chapitres 4 et 5
- Semaine 3 : chapitres 6 et 7
- Semaine 4 : chapitres 8 et 9

mais ces chapitres ne pourront pas être faits en entier. Sauf pour le premier TD, on consacrera au maximum 30 mn de TD au cours, en survolant certaines parties et en demandant aux étudiants de les lire (ils seront au programme du DS). Il sera possible d'y revenir aux TP et éventuellement aux TDs suivants.

J'ai voulu fournir beaucoup de cours pour que les étudiants aient un document de référence (notamment pour le C), mais il ne s'agit pas de tout faire !

L'idée est de faire lire le cours d'avance par les étudiants, de faire un parcours partiel et ensuite de répondre aux questions. Il s'agit aussi de faire faire aux étudiants des exercices variés. Les plus rapides pourront en faire plus. Je ne fournis pas de solutions. Si les étudiants souhaitent une solution, qu'ils demandent à ChatGPT. S'il reste du temps lors d'une séance, vous pouvez revenir à des exercices des TDs précédents.

Il est essentiel de rester en phase avec le planning. Si un chapitre n'est pas fini, c'est aux étudiants de finir de le lire, vous passez tout de même au chapitre suivant. Par contre, si à la fin du cours les étudiants veulent des explications complémentaires, prenez le temps de les leur donner. Incitez aussi les étudiants à suivre (chez eux ou en TP) les liens donnés, notamment vers les vidéos.

À part au premier TD, il faut se limiter à commenter les figures et photographies et à répondre aux questions. Ne faites pas plus d'une demi-heure de cours en deux heures, et fractionnez votre présentation. Évitez la lecture du support.

Pour les exercices, faites en priorité ceux marqués d'un astérisque, ensuite vous faites comme vous voulez/pouvez. Imposez cependant un ordre, sinon cela partira dans tous les sens. N'hésitez pas, si vous voulez, à utiliser vous-même ChatGPT pour les corrections. Je n'en fournis pas. Par ailleurs, le plus important n'est pas toujours la correction, mais le fait d'avoir des idées pour résoudre un problème.

5 Planning détaillé

SEMAINE 1 : chapitres 1, 2 et 3

TD1 :

- vous expliquez à vos étudiants le sujet du cours, vous distribuez les supports (actuellement dans mon bureau), vous donnez aussi le lien vers la version en ligne :

<https://tmpdocs.github.io/r103/r103-cours.pdf>

les étudiants doivent amener le support à chaque séance ;

- vous distribuez aussi le memento d'une page pour les étudiants ;
- vous indiquez le nombre d'heures (2 TDs et 1 TP chaque semaine, ce dernier en demi-groupe), vous indiquez que les TDs se font sans machine et sans téléphone et que l'usage d'un appareil, quel qu'il soit, pénalisera la note de contrôle continu ;
- vous indiquez aussi que dans chaque TD il y a une petite partie de cours, mais que les étudiants auront des choses à lire par eux-mêmes, et qu'ils pourront être interrogés sur des questions de cours lors du TD suivant ; signalez bien qu'il y a aussi des éléments historiques qui ne doivent pas être négligés, et qui sont importants pour le contexte du cours ; insistez sur le fait que vous êtes un guide et que les étudiants doivent fournir un travail personnel ; s'ils ne fournissent pas un minimum de travail personnel, ce sera l'échec assuré au DS ;
- expliquez le principe des TPs, où il s'agit de voir dans l'environnement Linux (si possible) l'application des notions vues en TD ;
- indiquez que lors de la troisième semaine il y aura une interrogation orale individuelle de 5mn ;
- décrivez succinctement l'environnement Linux, faites le lien avec le module d'introduction système, montrez un terminal, parlez de fichiers, de codage, de processus, etc., tout cela en 1-2 mn ; dites qu'on y reviendra en TP ;
- ensuite, en 30 mn, vous faites un survol de tout le support, en indiquant bien que les différents chapitres seront étudiés de manière plus approfondie dans les TDs à venir ; parlez notamment de codage, de portes logiques, d'IEEE754, etc.
- après le panorama général, revenez sur les chapitres 1 et 2, expliquez notamment bien les composants (figure 1.6), et les codes ASCII (figure 2.2) et UTF8 (figure 2.5) ;
- faites faire l'exercice 1 page 40, en faisant écrire aux étudiants des tableaux de nombres, en faisant les différences, et en faisant comprendre qu'on peut ainsi calculer les valeurs d'un polynome avec des additions répétées ; faites le lien avec Babbage ;
- passez le reste du temps sur UTF-8, avec des exercices de codage et de décodage ; les étudiants doivent bien faire la différence entre Unicode et UTF-8 ;
- si vous avez le temps, faites les exercices sur `base64` et `base45` ;
- s'il vous reste du temps, introduisez le C ;
- dites que ces notions seront vues en pratique pendant le TP ;
- enfin, **demandez aux étudiants de lire d'avance le chapitre 3** ainsi que les parties sautées des chapitres 1 et 2 ;

TD2 :

- repassez rapidement en revue le chapitre 3 (30 mn maximum) que les étudiants doivent avoir lu et répondez à leurs questions ; insistez sur les figures 3.2 à 3.5 et sur l'hexadécimal ;
- interrogez les étudiants sur des points particuliers des chapitres 1 et 2 ;
- puis passez à des exercices (je vous laisse choisir) ; l'essentiel est que les étudiants maîtrisent les conversions entre les bases 2 et 10, dans les deux sens, pour les entiers et les décimaux ;
- **demandez aux étudiants de lire d'avance le chapitre 4.**
- **donnez éventuellement un ou deux exercices du chapitre 3 à rédiger proprement à la main et à rendre pour la fois suivante**

TP :

- introduisez le C avec le chapitre 10 (seulement le minimum pour compiler et les bases) ;
- expliquez pourquoi on utilise le C, et assurez-vous que tous les étudiants soient en mesure de compiler des programmes C ; en attendant, ils peuvent éventuellement utiliser des compilateurs en ligne, comme <https://onecompiler.com/c> ;
- encouragez les étudiants à installer Linux ; si les étudiants veulent utiliser autre chose, on ne pourra pas forcément les aider ;
- faites des exercices faisant appel au C ou à d'autres langages, au choix. demandez aux étudiants de faire chez eux les autres exercices.
- vous pouvez notamment faire programmer les conversions entre base 2 et base 10.
- essayez aussi d'aborder le codage UTF8 au travers du C (accès aux bits d'un octet).

TD3 :

- passez en revue le chapitre 4 (≤ 30 mn); insistez sur les portes logiques et leur représentation, sur les tables de vérité, sur l'implémentation d'une table de vérité, expliquez les multiplexeurs, décodeurs, démultiplexeurs et additionneurs;
- interrogez les étudiants sur des points particuliers du chapitre 3;
- faites des exercices sur les circuits logiques (2, 3, 4, etc.);
- **demandez de lire d'avance le chapitre 5.**

TD4 :

- passez en revue le chapitre 5 (≤ 30 mn), en insistant sur la représentation des entiers sous forme naturelle et en complément à deux, en fonction du nombre de bits; expliquez l'intérêt du complément à deux pour les calculs, mais vous pouvez passer sur la multiplication et la division; expliquez directement IEEE754, mais laissez lire le détail préliminaire aux étudiants; dites que la norme IEEE754 contient aussi des cas particuliers, signalez essentiellement le codage de 0, de l'infini et de NaN;
- interrogez les étudiants sur des points particuliers du chapitre 4;
- faites des exercices (calculs en binaire, complément à deux, codage IEEE754);
- et **demandez de lire d'avance le chapitre 6.**
- **donnez éventuellement un ou deux exercices des chapitres 4 et 5 à rédiger proprement à la main et à rendre pour la fois suivante**

TP : exercices d'application, on pourra notamment aller sur des sites comme <https://nandgame.com>, <https://logic.ly/demo>, <https://circuitverse.org>, etc.; on pourra aussi utiliser des convertisseurs pour IEEE754 comme

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

TD5 :

- passez en revue le chapitre 6 (≤ 30 mn) essentiellement sur le langage machine; insistez sur le cycle de base (figure 6.1), la machine hypothétique (figures 6.2 et 6.3), les interruptions et les bus; montrez des extraits de codes de vrais processeurs et aussi au moins la figure 6.11 d'un code uniquement numérique de 1954;
- interrogez les étudiants sur des points particuliers du chapitre 5;
- faites des exercices (sur le simulateur du cours); si le temps manque, expliquez un vrai langage machine simple, genre Z80 ou 6502 (utilisez par exemple

<https://www.worldradiohistory.com/UK/Bernards-And-Babani/Babani/Babani-BP147-An%20Introduction-to-6502-Machine-Code.pdf>

ou <https://www.cs.jhu.edu/~phi/csf/slides/lecture-6502-intro.pdf>

puis un simulateur comme <https://itema-as.github.io/6502js>

L'objectif est de donner des notions sur le codage des instructions, sur les registres généraux et spécialisés, sur le registre d'état, sur la pile, etc.

- **demandez de lire d'avance le chapitre 7.**

TD6 :

- passez en revue le chapitre 7 sur la mémoire (≤ 30 mn), insistez sur la mémoire cache (figure 7.4), montrez sur la figure 7.6 la présence de plusieurs caches, détaillez les mémoires DRAM et SRAM et leur structure (figures 7.9, 7.10 et 7.11), la construction d'une ROM avec un décodeur, etc.; rappelez les unités binaires Kibit, Mibit, etc.

(cf. https://fr.wikipedia.org/wiki/Préfixe_binaire);

- interrogez les étudiants sur des points particuliers du chapitre 6;
- faites des exercices
- **demandez de lire d'avance le chapitre 8.**
- **donnez un ou deux exercices des chapitres 6 et 7 à rédiger proprement à la main et à rendre pour la fois suivante**

TP : exercices d'application, écriture de programmes en langage machine d'abord avec la machine simplifiée du cours, puis avec un simulateur de 6502 (ou autre), tests de désassemblage et exercices sur la mémoire en C; + INTERROGATION ORALE de 5mn

TD7 :

- passez en revue le chapitre 8 sur les processus (≤ 30 mn), en expliquant bien l'ordonnancement (figure 8.1), le *swapping* (figure 8.2), le partitionnement (figures 8.3 et 8.4), l'allocation de cadres (figure 8.5), la distinction entre adresses logiques et physiques (figure 8.6) et la mémoire virtuelle (figure 8.9);
- interrogez les étudiants sur des points particuliers du chapitre 7;
- faites des exercices (notamment sur la pagination)
- **demandez de lire d'avance le chapitre 9**, mais on n'aura pas le temps de le faire.

TP : exercices d'application et de révision pour le DS.

DS :

6 Indications pour les illustrations

Voici encore quelques indications pour les illustrations du cours :

attention, vérifier qu'il n'y a pas de décalage de numéro, j'ai au moins ajouté une figure quelque part

- les figures manuscrites sont toutes décrites dans le texte ; si nécessaire, je peux encore ajouter des précisions, mais je pense que c'est assez clair ;
- pour les photographies « historiques », voici ce que vous pouvez éventuellement dire en cours (vous n'êtes pas obligé de toutes les décrire, vous n'aurez sans doute pas le temps) :
 - fig 1.3 : Intel 4004, il me semble bon que les étudiants aient au moins une idée de la date (1971)
 - fig 1.4 : von Neumann, dites que c'était un mathématicien qui a touché à beaucoup de domaines
 - fig 1.5 : Harvard Mark I, notez la taille
 - fig 1.7 : attirez l'attention sur quelques composants comme l'UAL, le registre d'instructions, etc. ; et dites que le 6502 est le processeur qui a lancé Apple
 - fig 1.8 : Babbage, automatisation de calculs de tables par la méthode des différences finies, puis machine analytique
 - fig 1.9 : fragment de la première machine à différences
 - fig 1.10 : fragment de la machine analytique (jamais achevée, mais il y a actuellement un projet pour le faire)
 - fig 1.11 : machine analytique de Ludgate (jamais construite), les petites tiges servaient à stocker les valeurs
 - fig 1.12 : Torres Quevedo, a aussi conçu une machine analytique, un automate pour les échecs, et plein d'autres choses
 - fig 1.13 : analyseur différentiel de Bush, on est ici dans le calcul analogique
 - fig 1.14 : machine Z3 de Zuse, ordinateur à relais
 - fig 1.15 : machine à prédire les marées, exemple de machine qui construit une fonction complexe comme somme de fonctions sinusoïdales
 - fig 1.16 : un calculateur analogique de 1949, insister sur la différence entre analogique et numérique (ou digital, comme les gens disent aujourd'hui)
 - fig 1.17 : vous pouvez expliquer le fonctionnement d'un relais (pas très difficile !)
 - fig 1.18 : ce à quoi ressemblaient les relais de Zuse
 - fig 1.19 : exemple de lampe triode, le filament chauffé envoie des électrons qui sont recueillis par la plaque, mais le courant qui va du filament à la plaque est amplifié ou réduit par le potentiel de la grille ; la lampe triode sert donc d'amplificateur ;
 - fig 1.20 : une lampe soviétique !
 - fig 1.21 : des lampes dans l'ENIAC ; elles tombaient régulièrement en panne ;
 - fig 1.22 : un module de l'IBM 705 avec ses lampes ;
 - fig 1.23 : des transistors
 - fig 1.24 : encore des transistors (celui du milieu en haut est le fameux CK703)
 - fig 1.25 : le die du 8086 avec 29000 transistors
 - fig 1.26 : idem, mais on peut zoomer pour voir les transistors

- fig 2.3 : montrer la lecture de ces tables
- fig 2.6 : carte perforée 80 colonnes, montrer comment ça se lit
- fig 2.9 : métiers de Jacquard, montrer les cartes perforées qui programment les motifs de tissage
- fig 2.10 : je vous laisser décoder
- fig 3.1 : les puissances de 2 chez Leibniz vers 1703
- fig 3.6 : les triplets pythagoriciens de Plimpton 322
- fig 4.2 : l'ordinateur d'Apollo a beaucoup fait avancer les choses et on a ici un petit élément de cet ordinateur
- fig 4.22 : Aristote, parler un peu des origines de la logique (des syllogismes, genre Tous les hommes sont mortels, Socrate est un homme, donc Socrate est mortel) (c'est un syllogisme Barbara, cf. l'un des exercices)
- fig 5.11 : règle à calcul, vous pouvez expliquer le calcul de $\sqrt{2.1}$;
- fig 5.16 : les volumes de Knuth
- fig 5.17 : la vraie machine d'Anticythère (i.e., pas celle d'Indiana Jones)
- fig 5.18 : Pascaline, pour faire des additions avec un stylet (comme les anciens téléphones S63)
- fig 5.19 : machine de Leibniz, aussi pour des multiplications (on pouvait décaler la valeur à multiplier)
- fig 5.20 : arithmomètre Thomas, très courant à la fin du 19e siècle
- fig 5.21 : machine Odhner, ultra courante dans tous les bureaux jusque vers 1960; (un tour de manivelle = une addition)
- fig 5.22 : une machine Friden, je crois le modèle qui pouvait calculer les racines carrées (à vérifier)
- fig 5.23 : la célèbre calculatrice de poche Curta
- fig 6.9 : L'IBM PC, un peu oublié aujourd'hui, mais révolutionnaire à l'époque
- fig 6.10 : L'un des ancêtres des processeurs RISC
- fig 6.11 : un programme tout numérique
- fig 6.12 : un programme pour IBM 650, vraiment de l'assembleur
- fig 6.13 : exemple en FORTRAN, vous pouvez expliquer le code
- fig 7.6 : hiérarchie mémoire de l'i7, montrer simplement qu'il y a plusieurs caches et la mémoire principale
- fig 7.7 : tambour de mémoire, qui tourne grâce au moteur de droite!
- fig 7.8 : un module de 1024 core, je pense construit à la main! voyez ici : <https://www.youtube.com/watch?v=paQ3zIsz1-8> c'est tout petit, chaque petit tore stocke un bit;
- fig 7.25 : les trois anciens formats de disquettes, en haut 8.5" (que je n'ai vu que de loin), en bas à droite 5"1/4 (que j'ai beaucoup utilisé), puis 3"1/2, que nous avons tous connus;
- fig 7.31 : les armoires avec une bande qui se déroule d'un disque à l'autre, comme les cassettes audio (les étudiants connaissent-ils les cassettes audio?)
- fig 7.33 : programmation de l'ENIAC en déplaçant manuellement des fiches
- fig 9.3 : l'UNIVAC 1 dans sa salle toute propre et tous ses accessoires
- fig 9.4 : là où on entrait un programme dans l'IBM 650, pas d'écran...
- fig 9.5 : imprimante matricielle
- fig 9.6 : interface de l'ordinateur (embarqué) d'Apollo; c'était minimaliste (AGC = Apollo Guidance Computer)
- fig 10.2 : les auteurs du C, devant le PDP-11, sans écran